Real Time SDN DDoS Detection and Mitigation System

BCSE308P - Computer Networks

PROJECT BASED COMPONENT REPORT

by

_____

Name and Register Number of Candidate(s)

School of Computer Science and Engineering



Submission Date: _____

DECLARATION

I hereby declare that the report entitled "Real Time SDN DDoS Detection and Mitigation System" submitted by me for BCSE308P - Computer Networks to VIT is a record of bonafide work carried out by me under the supervision of _____.

I further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for any other courses in this institute or any other institute or university.

Place: _____

Date: _____

Signature of the Candidate(s): _____

TABLE OF CONTENTS

Contents	Page No.
1. Introduction	4
2. Problem Statement	4
3. Objectives	4
4. Literature Review and Existing System	5
5. Abstract of the Proposed Project Work	5
6. Workflow of the Proposed Project Work and System Architecture	5
7. Contribution of Individual Team Members	6
8. Modules Explanation	6
9. Project Implementation	6
a. Experimental Setup with Hardware and Software Requirements	6
b. Test Cases	7
c. Results of Execution	7
d. Performance Analysis	8
e. Critical Evaluation and Optimization	8
10. Results and Discussion	9
a. Findings and Inferences	9
b. Innovation, Novelty, and Evidence	9
11. Conclusion	9
12. Limitations and Future Enhancement	9
References	10

1. Introduction

Distributed denial of service (DDoS) traffic comes from multiple sources and can reduce the availability of a network service. Software defined networking (SDN) places forwarding decisions under a programmable controller, which can observe switch statistics and change forwarding rules while traffic is flowing. This project develops a controlled, single-switch laboratory system for detecting a UDP flood and applying short-lived, source-specific blocks. Its purpose is to show the complete path from traffic observation to a measurable service outcome [1-3].

The implemented system includes the private Mininet topology, UDP echo service, probe client, OS-Ken OpenFlow 1.3 controller, per-source counter monitor, bounded flood generator, sustained-rate detector, and temporary selective blocks. M2-M6 tests and matched attack trials were completed on 13 October 2026. Raw switch, victim, client, and controller logs support the results below [6-10].

2. Problem Statement

A victim UDP service may receive packets from several sources faster than it can handle them. A network administrator needs to distinguish a distributed flood from ordinary variation, identify the contributing sources, block only malicious traffic, and preserve legitimate access. A generic switch rule that combines all UDP senders hides the per-source counters needed for attribution. Blocking every packet to the victim can also make the defense as disruptive as the flood itself.

The technical problem is therefore to use attributable OpenFlow counters, a reproducible decision rule, and precise temporary drop rules in an isolated lab. Success must be judged by both attack reduction at the victim and the continued success of a legitimate client probing the same service. An alert by itself is insufficient evidence of effective mitigation [6].

3. Objectives

No.	Objective	Evidence or acceptance target
1	Build one-switch, five-host private topology with a UDP echo victim.	20/20 pings and 10/10 baseline probes passed in M2.
2	Forward ARP, ICMP, and UDP through OS-Ken; retain per-source UDP counters.	M3 passed 20/20 pings, 16/16 initial UDP probes, six counted UDP flows.
3	Detect a sustained two-source UDP flood using counter deltas.	1.555 s alert; both sources identified in matched trial.
4	Apply temporary source-and-destination-specific UDP blocks.	Two exact 10 s rules confirmed; both expired after attack.
5	Measure benefit and collateral effect in matched trials.	100% lower post-block attacker rate; client 80/80. No measured availability gain.

The observed results in objectives 3-5 are from the matched 600-packet/s-per-source trial; the post-block reduction excludes detection time [10].

4. Literature Review and Existing System

Mininet creates virtual hosts, switches, links, and controllers that run standard Linux networking code, making it suitable for repeatable SDN experiments [1]. OpenFlow 1.3 defines flow tables, counters, controller messages, and rule timeouts that support feedback-based control [2]. OS-Ken exposes flow-statistics requests and replies to a Python controller [3]. Open vSwitch provides the switch implementation used in this lab [4].

Prior research has already demonstrated SDN-based flood defense. FloodDefender, for example, addresses denial-of-service pressure on both the SDN data and control planes [5]. FlowGuard does not claim a new DDoS algorithm. Its course-level contribution is a reproducible five-host implementation with per-source attribution, legitimate probes to the attacked service, bounded attack generation, matched off/on trials, and inspectable raw data [6-10].

The earlier forwarding baseline created source/destination-specific UDP flows with identifiable cookies and nonzero counters [8]. The finished controller polls those counters, detects a sustained two-source flood, installs narrowly matched drop rules, and verifies switch acceptance with a barrier reply [9, 10].

5. Abstract of the Proposed Project Work

FlowGuard implements a real-time SDN DDoS detector and mitigator in a private Mininet network. Two attack hosts and one legitimate client reach a UDP echo victim through Open vSwitch. An OS-Ken controller polls per-source OpenFlow counters every second, computes reset-safe rates, and requires two high-rate sources plus two consecutive high aggregate-rate windows before applying 10-second source-specific drop rules. In matched eight-second trials at 600 packets/s per attacker, the alert appeared after 1.555 s and the switch confirmed rules after 1.634 s. Comparable complete victim windows after confirmation fell from 1,199.956 to 0 attacker packets/s. The legitimate client received 80/80 replies with mitigation, but also 80/80 without it, so an availability improvement was not demonstrated [9, 10].

6. Workflow of the Proposed Project Work and System Architecture

All generated traffic remains within Mininet. The victim is 10.0.0.10:9000; the legitimate client is 10.0.0.11; possible attack hosts are 10.0.0.21, 10.0.0.22, and 10.0.0.23. Each host connects to switch s1. The controller listens on 127.0.0.1:6653 and programs the switch over OpenFlow 1.3 [6, 7].

Traffic and control path	**Function**
Client and attack hosts -> s1 -> victim	Data plane: UDP packets to the echo service.
s1 -> OS-Ken controller	Control plane: packet-in events and one-second flow-statistics replies.
Controller -> s1	Forwarding rules and temporary attacker-specific drop rules.

The implemented monitor filters the controller's UDP forwarding-flow cookie, calculates each source's packet and byte rates from counter differences and elapsed monotonic time, and aggregates victim rates. First samples, missing flows, and counter resets produce no false spike. Thresholds are 100 victim packets/s and 30 packets/s for each of at least two sources, sustained for two windows. The controller adds priority-200 rules matching UDP source, victim, and port 9000; a barrier reply confirms switch installation. Rules have a 10-second hard timeout; ARP and the legitimate client's separate flow remain permitted [9, 10].

7. Contribution of Individual Team Members

Complete this section before submission. Record each person's actual work and ensure that the declaration and cover list the same candidates.

Name and register number	Actual contribution	Evidence or file
_____	_____	_____
_____	_____	_____
_____	_____	_____
_____	_____	_____

8. Modules Explanation

Module	State	Function
Topology and service	Implemented	Five fixed Mininet hosts, Open vSwitch s1, UDP echo server, and legitimate probe.
OpenFlow forwarding	Implemented	OS-Ken learns ports, forwards ARP/ICMP/UDP, and installs per-source UDP rules.
Counter monitor	Implemented	Poll packet/byte counters each second and compute reset-safe source rates.
Detection	Implemented	Require two high windows and two high-rate sources before alerting.
Mitigation and recovery	Implemented	Install exact UDP drop rules with 10 s hard timeout; observe removal.
Experiment runner	Implemented	Generate bounded lab-only load; save controller, victim, client, and switch logs.

9. Project Implementation

9a. Experimental Setup with Hardware and Software Requirements

The verified execution environment is Ubuntu 26.04 LTS in WSL2, Linux kernel 6.18.33.2-microsoft-standard-WSL2, Python 3.14.4, Mininet 2.3.0-1.1build1, Open vSwitch 3.7.1-2, and OS-Ken 4.1.1-1ubuntu2. WSL reported an Intel Core i5-12450HX, 12 logical CPUs, and 7,981,708 kB guest memory [10].

The single switch and five fixed private hosts have no NAT or external link. The controller runs secure-mode OpenFlow 1.3. Matched attacks use two senders at 600 packets/s each for eight seconds with 128-byte application payloads and a client echo probe every 100 ms. The generator is restricted to the victim endpoint, at most 2,000 packets/s per sender and 30 seconds per run. Controller polling is once per second; the victim, probe client, controller, and switch each save timestamped evidence [9, 10].

9b. Test Cases

Case	Procedure	Status
M2 topology	Ping every host pair; send 10 echo probes; confirm cleanup.	Passed, recorded
M3 forwarding	Ping host pairs; send 16 initial echo probes; inspect six UDP flows.	Passed, recorded
M3 reconnect	Restart controller; check stale-flow removal and three new probes.	Passed, recorded
M4 detection	Run five-minute benign baseline and two-source attack; inspect alert timing.	Passed, recorded
M5 mitigation	Inspect exact drop rules, legitimate traffic, expiry, and second wave.	Passed, recorded
M6 comparison	Match 600-packets/s-per-source attack with mitigation off and on.	Passed, recorded

9c. Results of Execution

M2 and M3 verified topology, UDP service, forwarding, attributable counters, and controller restart [7, 8]. Nine focused unit tests pass, covering forwarding, counter reset and missing-flow handling, short-burst rejection, client protection, and exact drop matches [9]. The later trials recorded the following results from controller events, victim receive logs, client CSV, generator records, and switch flow dumps [9, 10].

Trial	Observed execution result
M4 benign baseline	Five minutes; client 600/600, second benign sender 600/600; 307 samples; zero alerts and drop requests.
M4 detection	Two 600-packet/s senders identified; alert after 2.305 s; client 80/80; mitigation disabled.
M5 mitigation	Alert after 2.061 s, switch barrier after 2.131 s; two exact drop rules; post-block attacker rate 0; client 80/80.
M5 repeat	Two separated attack waves; two alerts, four drop requests and removals; client 80/80 in each wave.
M6 matched	Off: 1,199.956 attacker packets/s; on: 0 after block. On alert 1.555 s, barrier 1.634 s; client 80/80 in both.

The M5 switch flow-table sample counted 1,000 and 916 packets on the two drop rules, which confirms that the rules matched flood packets. Both rules later disappeared. In M6, the rules expired about 3.6 seconds after attack end, and no attack drop rule remained in the recovery dump [9, 10].

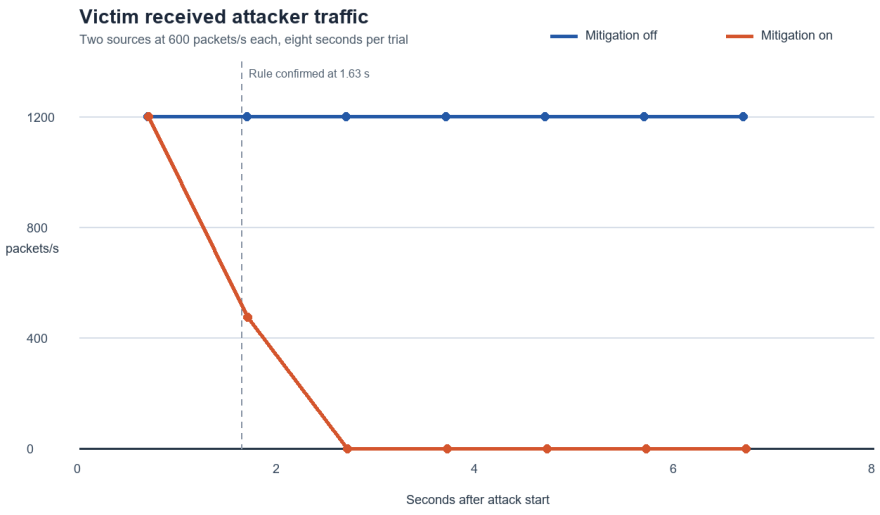


Figure 1. Victim-received attacker packets in the matched M6 trials. Data: saved one-second victim logs; the zero-rate region follows switch confirmation [10].

The repository contains timestamped raw logs, CSV probe records, JSON summaries, flow dumps, and this derived chart. No original execution screenshots were captured; any screenshot requirement for final submission needs genuine images from a reproduced run.

9d. Performance Analysis and Comparison of Methods

The off/on comparison held topology, two attacker addresses, 600 packets/s per attacker, eight-second duration, 128-byte payload, and client cadence constant. Post-block rates compare complete one-second victim windows beginning at least 0.5 seconds after the mitigation barrier with the same relative windows in the disabled trial. Thus the 100% reduction describes the steady post-block period, not the entire attack [10].

Metric	Target	Measured M6 result
Alert latency from generator start	<=5 s	1.555 s; barrier confirmation 1.634 s
False alerts / blocks in five-minute baseline	0	0 / 0 across 307 samples
Post-block victim attacker rate	>=90% reduction	1,199.956 off vs 0 on packets/s; 100% lower
Whole-trial full-window attacker rate	Report separately	1,200.023 off vs 239.244 on packets/s; 80.1% lower
Legitimate echo success with mitigation	>=95%	80/80 (100%); also 80/80 without mitigation
Drop-rule recovery	<=15 s	Both expired about 3.6 s after attack end

Latency uses the generator start record immediately before its first packet, so it approximates first-packet latency. The 80.1% whole-trial reduction includes the detection and installation interval. The client succeeded on every probe in both conditions; the experiment cannot support a claim that mitigation improved service availability [10].

9e. Critical Evaluation and Optimization

Per-source UDP flows permit attribution without inspecting payloads. Filtering the app's forwarding cookie limits measurement to relevant counters. The reset-safe monitor avoids false spikes from first samples, expired flows, and counter resets. A short simultaneous benign burst reached about 39.929 victim packets/s and caused no alert, whereas the sustained two-source flood crossed the 100-packets/s victim and 30-packets/s source thresholds in consecutive windows [9, 10].

The exact UDP source/victim/port match protected the client while stopping both attacker sources; barrier replies and drop counters support installation. A repeated-wave trial checked expiration and re-detection. The fixed thresholds are calibrated only for these local traffic patterns. One switch and one WSL host do not model a congested uplink or overloaded service, so selective filtering is the supported conclusion [9, 10].

10. Results and Discussion

10a. Findings and Inferences

The end-to-end path was observed: flow counters rose for two sources; the controller identified them after two high windows; the switch confirmed two exact drop rules; victim logs showed zero attacker packets in comparable complete post-block windows; and the legitimate client retained 80/80 replies. Normal traffic and a short benign two-source burst produced no alerts. Two separated attack waves produced two alerts and no persistent duplicate rules [9, 10].

Unmitigated service also returned 80/80 client replies at 1,200 aggregate attacker packets/s. A bounded exploratory run at 2,000 packets/s from each attacker still returned 80/80 without mitigation. Consequently the data show effective selective packet filtering, not improved service availability under this lab load [10].

10b. Innovation, Novelty, and Evidence

The course demonstration is an auditable closed loop: the client probes the same service being flooded; the controller attributes traffic by source; and temporary rules match only the identified UDP sources and target. Timestamped raw controller, generator, victim, client, and flow-table files plus a deterministic analyzer permit recalculation of the reported comparisons. These are implementation and evaluation choices built from established SDN methods, not a new detection algorithm [2, 5, 9, 10].

11. Conclusion

FlowGuard completed and tested topology, OpenFlow forwarding, per-source monitoring, two-source flood detection, precise temporary mitigation, repeated-wave recovery, and matched off/on trials. At 600 packets/s per attacker, it alerted within 1.555 seconds and cut victim-received attacker traffic by 100% in the comparable post-block period while the client received 80/80 replies. The same client also received 80/80 replies without defense, so the experiment demonstrates selective filtering and preservation of access but not an availability improvement [7-10].

12. Limitations and Future Enhancement

The prototype uses one switch, fixed private addresses, one UDP service, a known five-host topology, and two controlled attack senders. It shares one WSL host rather than an independent bottleneck or internet path. Generator start is a proxy for first-packet time; the baseline covers five minutes and one short burst, not every benign traffic pattern. The trials cannot quantify availability benefit, internet-scale behavior, spoofed-source robustness, or general false-positive rates. Original screenshots were not captured [10].

Future evaluation could introduce a measured service or link bottleneck so the unmitigated attack measurably harms legitimate probes, then repeat matched off/on trials. Additional work could test more runs and confidence intervals, TCP SYN and spoofed-source traffic, multiple switches, adaptive thresholds, and genuine execution screenshots for the final submission [6, 10].

References

[1] Mininet, "Mininet Overview." https://mininet.org/overview/ (accessed 13 October 2026).

[2] Open Networking Foundation, OpenFlow Switch Specification, Version 1.3.4, 2014. https://opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.3.4.pdf

[3] OpenStack, "OS-Ken OpenFlow v1.3 Messages and Structures." https://docs.openstack.org/os-ken/latest/ofproto_v1_3_ref.html (accessed 13 October 2026).

[4] Open vSwitch, "OpenFlow FAQ." https://docs.openvswitch.org/en/stable/faq/openflow/ (accessed 13 October 2026).

[5] S. Gao, Z. Peng, B. Xiao, A. Hu, and K. Ren, "FloodDefender: Protecting data and control plane resources under SDN-aimed DoS attacks," IEEE INFOCOM, 2017. https://ieeexplore.ieee.org/document/8057009

[6] FlowGuard project plan and task tracker, local project archive: docs/PROJECT_PLAN.md and docs/TASKS.md, reviewed 13 October 2026.

[7] FlowGuard M2 smoke-test evidence and setup guide, local project archive: evidence/m2/SMOKE.md and docs/SETUP.md, 7 October 2026.

[8] FlowGuard M3 controller smoke-test evidence and controller guide, local project archive: evidence/m3/SMOKE.md and docs/CONTROLLER.md, 11 October 2026. Repository: https://github.com/Sai181006/FlowGuard

[9] FlowGuard M4 and M5 recorded trial evidence, local project archive: evidence/m4/RESULTS.md and evidence/m5/RESULTS.md, 13 October 2026.

[10] FlowGuard M6 results, raw trial records, and analyzer, local project archive: docs/RESULTS.md, results/m6-attack-off/, results/m6-attack-on/, results/m6-summary.json, and experiments/analyze.py, 13 October 2026.